TacticalMap: The Operational Edge in the Absence of Connectivity

Computer Engineering

Project work of Mobile and Social Sensing System



Manages a Team in **danger** situation:

- **Zero-Connectivity Ready:** Native offline mapping & P2P Bluetooth Communication.
- **Smart Data Pipeline:** Adaptive GPS tracking for battery and network optimization.
- **Hands-Free SOS:** "Back Tap" gesture for rapid emergency alerts.

Team members:

Klaudio Ciacia
Gabriele Caioli
Matteo Halilaga
Lorenzo Vanni

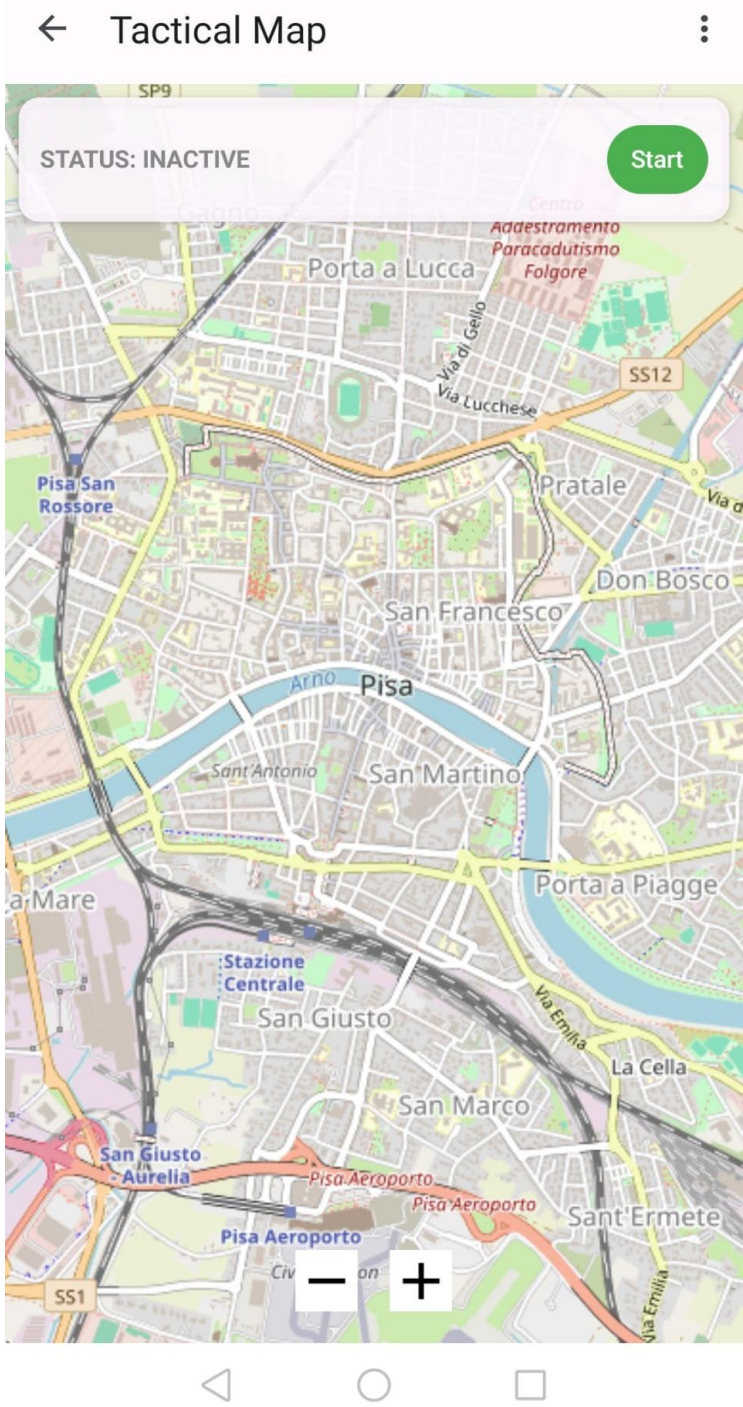
TacticalMap: Osmdroid

Why osmdroid?

1. Tactical Focus: Native **offline caching** support for operations in zero-connectivity or remote environments.

2. **Open** Standard: Based on OpenStreetMap (OSM), ensuring total independence from proprietary vendors and API restrictions.

3. Custom **Graphics**: Flexible layered architecture ideal for tactical symbology (e.g., SOS pulse animations, real-time soldier path tracking).



TacticalMap: User Interface

1. **Marker**: green for status_ok, red for status_danger.
2. **Label** on top of marker to describe teammate ID, status, activity and timestamp.
3. **SOS** pulse animation for danger situation.
4. **Historical path** colored in red for danger, green for status_ok and blue for other teammates (status_ok).
5. **Command & Control** panel: visualization of status, button to start/end mission, current duration.

Policy Engine

	ACTIVITY	GPS INTERVAL	SMALLEST DISPLACEMENT
BATTERY > 30%	STILL	120 sec	20 m
	WALKING	30 sec	10 m
	RUNNING	10 sec	5 m
BATTERY < 30%	STILL	240 sec	30 m
	WALKING	60 sec	30 m
	RUNNING	20 sec	30 m
DANGER	EVERYTHING	10 sec	0 m

STILL > 60 min → Emergency danger activated

Smallest Displacement & Data Slimmer

SmallestDisplacement (Google Play Services, Hardware**)**

- **Smart Monitoring:** Utilizes device sensors (e.g., accelerometer).
- **Distance Trigger:** Detects movement exceeding a specific threshold (e.g., > 20 meters).
- **Efficiency:** Wakes up the app to update coordinates only when necessary.

DataSlimmer.kt (Filters more traffic data for Firebase database, Software**)**

- **Deduplication**: Blocks redundant or inaccurate data (like GPS drift when the phone is stationary, indoor) protecting the backend from useless traffic.
- **Absolute Priority** (DANGER Status): Emergencies bypass all filters. The record is transmitted immediately.
- **Activity Change**: If the user switches contexts (e.g., from STILL to RUNNING), the system instantly syncs the new state.

LocationTracker

- **Source**: **FusedLocationProviderClient** (Best-effort energy source, GPS, Wi-Fi, Cell Tower).
- **Data Pipeline**: Raw Coordinates ➔ DataSlimmer Evaluation ➔ Room DB Insertion ➔ Firebase Auto-sync.
- **Coroutine Management**:
  - PolicyEngine.currentConfig: listen to PolicyEngine. Dynamically reapplies **GPS** configurations (e.g. 60s → 10s).
  - PolicyEngine.policyState: Intercepts the **DANGER** state to **force** immediate data transmission on Firebase without waiting for timout.
  - StartFirebaseSync: runs every 120s to sync **pending records** marked as unsynced in the local database because of offline connection for long period.

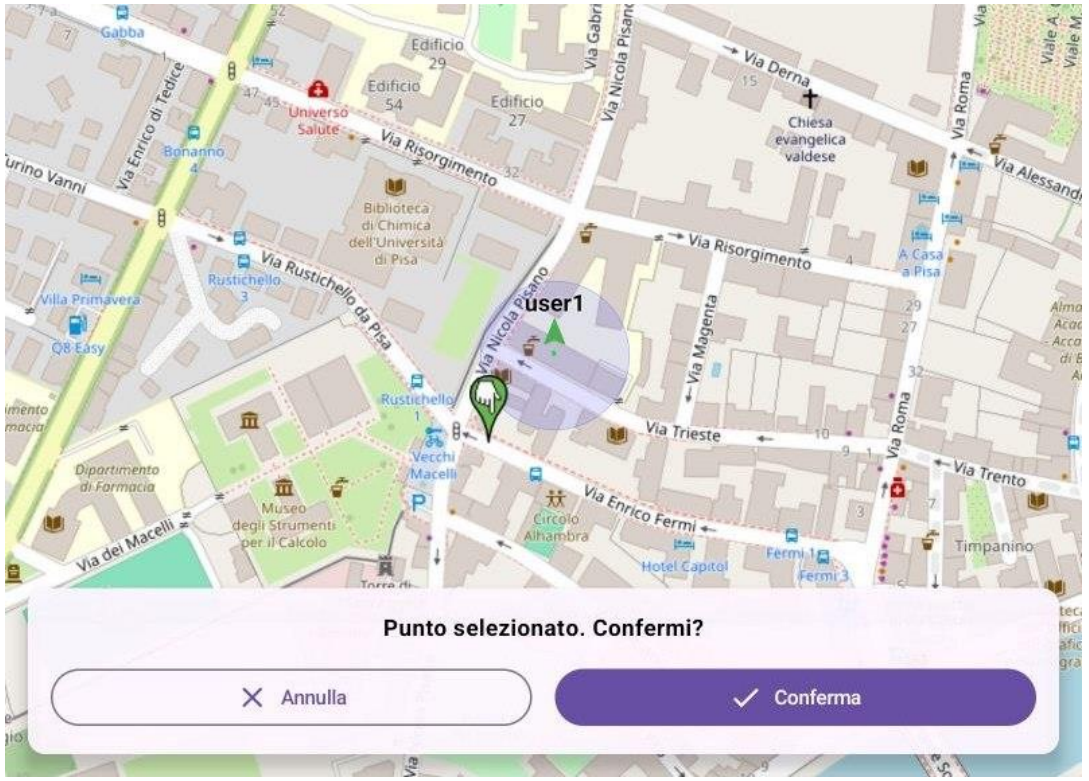
Tactical Offline Mapping

AREA OF OPERATION (AO) SETUP

Defining the AO

The "Map Picker" allows coordinators to define the central point of a mission.

- → **Firebase Sync:** Real-time distribution of AO coordinates to the entire team.
- → **Truth at Download:** Explicit verification before downloading ensure latest coordinates are used.
- → **WorkManager:** Background processing continues even if the app is closed.



OFFLINE RESILIENCY

Comparison of Map Storage Methods:

Feature	Standard osmdroid Cache	SQLite Archive
Compilation	Passive: requires users to manually scroll around the map	Active/Explicit: pre-fetched via CacheManager
Persistency	Volatile: overwritten by LRU logic as the cache fills	Permanent until manually deleted
Reliability	High risk of missing tiles (blank screens)	Guaranteed AO Tiles

Covers a 1km radius (Zoom 4-19) for maximum tactical detail.

Emergency SOS Gesture

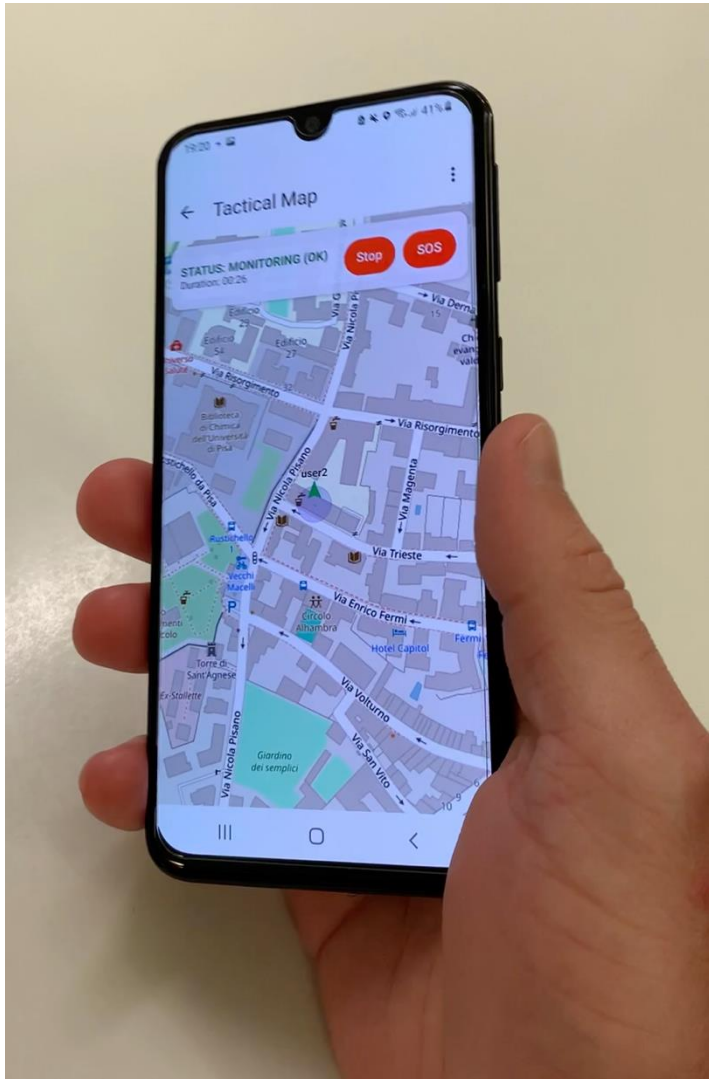
THE TRIPLE-TAP SOLUTION

Challenge

In high-stress tactical situations, interacting with a touchscreen is often slow, difficult, or impossible due to gloves or limited visibility.

Solution

Implementation of a **"Back Tap Gesture"**. Using internal sensors to detect three rapid taps on the device rear, bypassing the UI entirely. Requires active tracking. Functions even with the screen off.



TECHNICAL SIGNAL PROCESSING

Jerk Detection

Monitors Delta-Z spikes in linear acceleration to isolate sharp taps from general movement.

$$\Delta Z = | Z_n - Z_{n-1} |$$

Axial Dominance

Verifies the impulse is strictly perpendicular to the screen (Z-axis > 1.5 (X+Y)) to prevent false positives.

Temporal Window

Triple-tap validation requires events within 1000ms, with a 250ms cooldown to avoid vibration loops.

**Powered by Linear Acceleration sensors (ignoring gravity) for 3D orientation independence.*

Offline communication

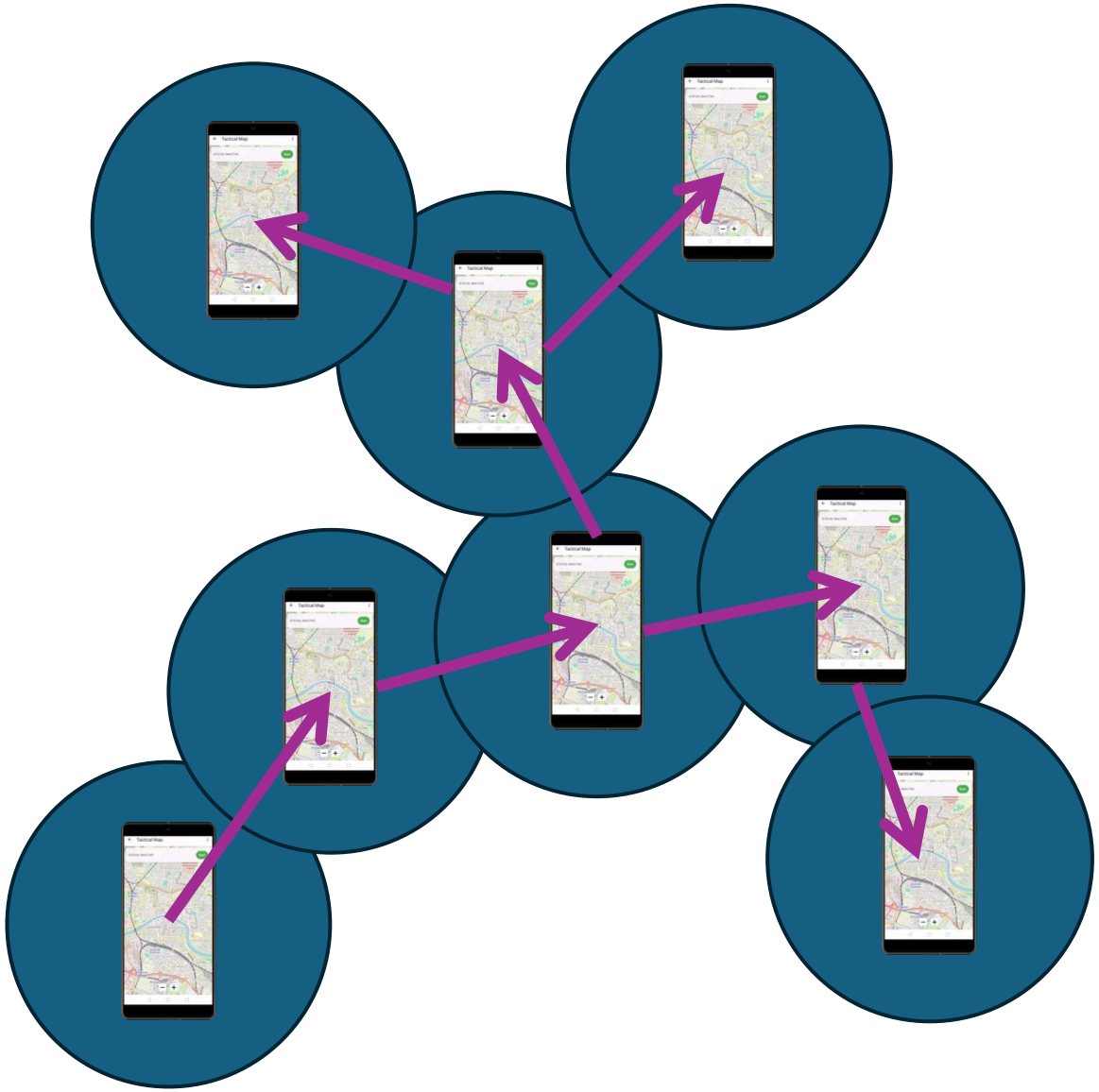
MESH NETWORK

Challenge

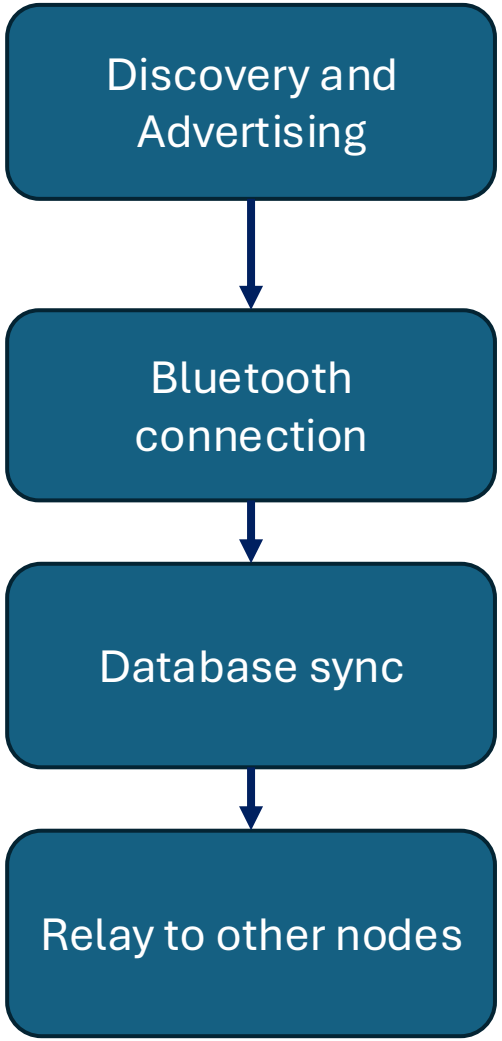
In some critical scenarios internet infrastructure might be compromised but first responders on the scene still need to coordinate with each other

Solution

Implementation of peer to peer communication using a Bluetooth mesh network to communicate with nearby offline nodes that will spread the information to their neighbors



P2P CONNECTION



- **Discovery and Advertising**

- Advertising uses BLE
- Discovery intervals (adaptive):
  - Normal: 30s
  - Low battery: 2 min
  - Danger status: 10s

- **Database sync**

- Nodes track last sync timestamp per peer
- Request/transfer only records created after that timestamp
- Merge incoming records only if newer

- **Relay**

  - Queue received records for retransmission
  - Send to connected peers (except sender)
  - Timing: DANGER (immediate) → Normal (2s) → Low battery (skip)

	Bluetooth + BLE	Wi-fi direct	Google Nearby Connections API
Energy usage	Bluetooth: 20-50 mA BLE: 5-15 mA	50-150 mA per connection	15-80 mA
Range	10-50 m	60-200 m	10-200 m
Scanning frequency	Configurable	Fixed (10s)	Not configurable (5-10s)

(Deprecated)

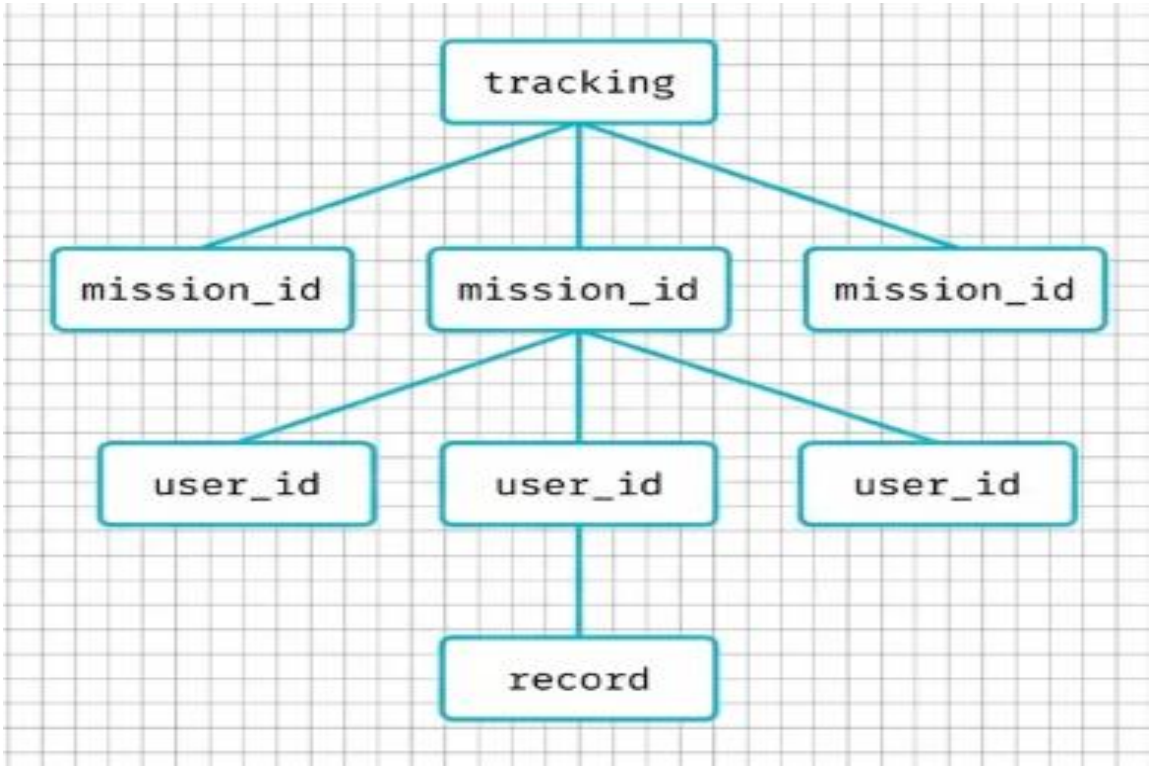
Local Persistence: Room

- **Data Modeling:** Use of the TrackingRecord data class (Entity) to natively map SQLite table schemas.
- **Compile-Time Safety (TrackingDao):** DAO interface for SQL query abstraction; Room validates syntax at compile time, ensuring high software resilience.
- **SingletonPattern(AppDatabase):** Central ized access via a class extending RoomDatabase.
  The **companion object** ensures a unique, thread-safe instance.

```
19  data class TrackingRecord(
20      @PrimaryKey(autoGenerate = true)
21      @get:PropertyName( value = "id") @set:PropertyName( value = "id")
22      var id: Long = 0,
23
24      @get:PropertyName( value = "missionId") @set:PropertyName( value = "missionId")
25      var missionId: String = "",
26
27      @get:PropertyName( value = "userId") @set:PropertyName( value = "userId")
28      var userId: String = "",
29
30      @get:PropertyName( value = "timestamp") @set:PropertyName( value = "timestamp")
31      var timestamp: Long = 0,
32
33      @get:PropertyName( value = "latitude") @set:PropertyName( value = "latitude")
34      var latitude: Double = 0.0,
35
36      @get:PropertyName( value = "longitude") @set:PropertyName( value = "longitude")
37      var longitude: Double = 0.0,
38
39      @get:PropertyName( value = "status") @set:PropertyName( value = "status")
40      var status: TrackingStatus = TrackingStatus.OK,
41
42      @get:PropertyName( value = "activityType") @set:PropertyName( value = "activityType")
43      var activityType: ActivityType = ActivityType.STILL,
44
45      @get:PropertyName( value = "isCloudSynced") @set:PropertyName( value = "isCloudSynced")
46      var isCloudSynced: Boolean = false,
47
48      @get:PropertyName( value = "lastMeshSyncTime") @set:PropertyName( value = "lastMeshSyncTime")
49      var lastMeshSyncTime: Long = 0
50  )
```

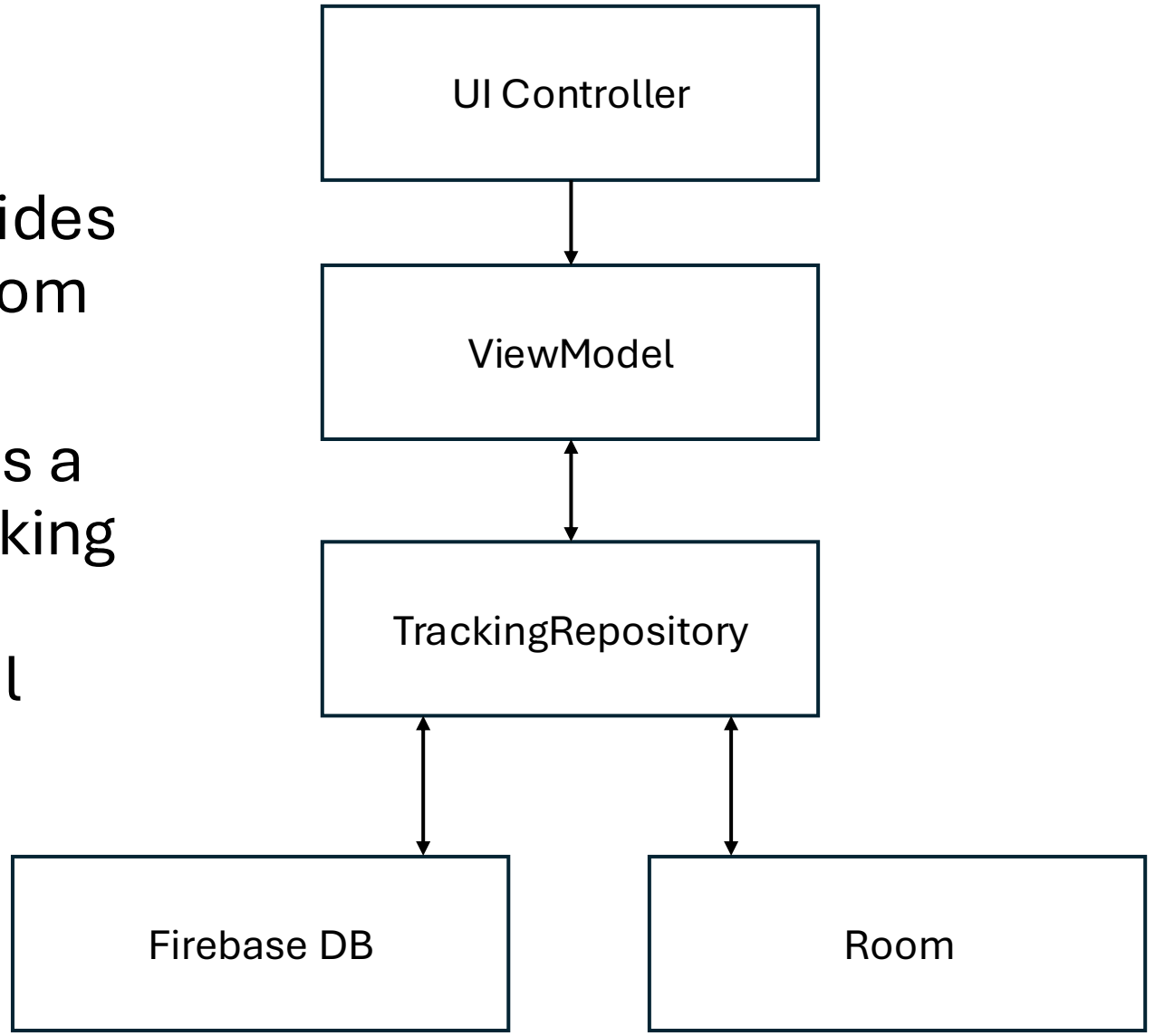
Remote Storage: Firebase Realtime Database

- Real-time synchronization
- Easy to use
- Integration with the Android ecosystem



Repository Pattern

- **UI Decoupling:** Abstracts and hides data management complexity from high-level components.
- **Source Independence:** Provides a single, centralized interface, making it transparent whether the information originates from local storage or the cloud.

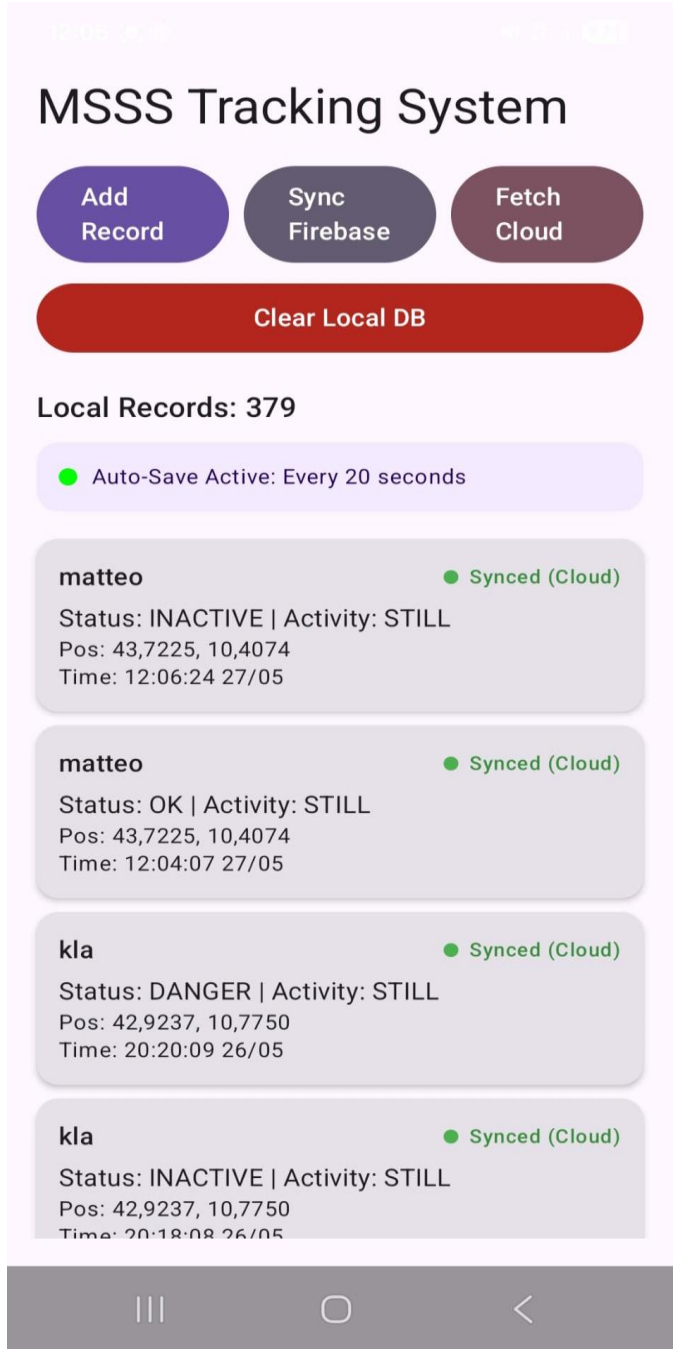


Code Optimization

- **Asynchronous Programming (Coroutines & Suspend Functions):** Allows delegating I/O operations to background threads, enabling the main thread to remain constantly responsive. Execution is suspended and resumed without ever "freezing" the user interface.
- **Reactive Architecture (Kotlin Flow):** Guarantees real-time, bidirectional synchronization between the local database and the map, triggering automatic graphical updates upon any data variation.
- **Event-Driven Synchronization (Callbacks & Listeners):** Decouples data reception using the observer pattern. It leverages LocationCallback to intercept GPS sensor state changes and Firebase ValueEventListeners to immediately receive new data from the cloud.

Test Interface

- A diagnostic interface (excluded from the production build) was developed to monitor in real time the status of database records, the consistency of map markers, and synchronization alignment with Firebase



Prototype Limitations & Future Developments

- **Realtime Database Bottlenecks:** Due to the high frequency of user location updates, the JSON tree structure of Firebase Realtime Database reveals architectural limitations.
- **Communication Overhead:** Since shallow queries (shallow fetches on deep nodes) are not supported, the SDK is forced to download the entire historical sub-node of positions just to retrieve a user's latest record. This generates a network overhead that is unsustainable at scale.
- **Migration to Cloud Firestore:** The definitive solution for production deployment involves migrating to Cloud Firestore which allows flat, flexible queries, isolating individual records.